Cross-site scripting (XSS)

Instrukcja omawia podatność cross-site scripting (XSS) w prostej aplikacji sklepu internetowego. XSS polega na wstawieniu danych wpisanych przez użytkownika do strony w taki sposób, że przeglądarka traktuje je jako kod (HTML lub JavaScript) i wykonuje. Ćwiczenie pokazuje trzy odmiany tej podatności: odbitą (ang. *reflected*), trwałą (ang. *stored*) oraz opartą na modelu DOM (ang. *DOM-based*). Dla każdej odmiany instrukcja prowadzi od wykrycia, przez wykorzystanie, do wersji poprawionej, odpornej na atak.

Model DOM (Document Object Model) to drzewiasta reprezentacja strony, którą przeglądarka buduje z kodu HTML i którą skrypty mogą odczytywać oraz zmieniać.

Po zakończeniu ćwiczenia student potrafi:

- wskazać miejsce, w którym dane użytkownika trafiają do strony bez kodowania,
- wykorzystać XSS odbity przez spreparowany adres URL,
- wykorzystać XSS trwały przez treść zapisaną w aplikacji i wykonywaną u innych użytkowników,
- wykorzystać XSS oparty na modelu DOM w wariancie, w którym kod atakującego nie trafia na serwer,
- wskazać poprawkę właściwą dla danego kontekstu (kodowanie wyjścia, `textContent`) i sprawdzić, czy usuwa ona podatność,
- opisać znaleziska i przebieg testu w formie przyjętej w raporcie z testu penetracyjnego.

Test penetracyjny to kontrolowana próba ataku na system, prowadzona w celu wykrycia podatności. Wymaga on pisemnej zgody właściciela systemu, określającej zakres i termin testów. Te same czynności wykonane bez zgody mogą stanowić przestępstwo (art. 267 Kodeksu karnego). Ćwiczenie dotyczy wyłącznie lokalnej aplikacji pod adresem `http://127.0.0.1:8000`, uruchomionej na własnym komputerze. Opisane techniki wolno stosować wyłącznie wobec własnych systemów lub za pisemną zgodą właściciela. Nie wolno ich kierować na inne cele, w szczególności na sieci i systemy AGH. Nie wolno też wysyłać innym osobom adresów ani treści z ładunkiem z tego ćwiczenia, ponieważ kod wykonałby się w przeglądarkach osób, które nie wyraziły zgody na test.

Spis treści

0. Wymagania 3
1. Jak działa XSS 4
2. Uruchomienie aplikacji 6
3. XSS odbity (reflected) 7
4. XSS trwały (stored) 10
5. XSS oparty na modelu DOM (DOM-based) 12
6. Poprawki 15
7. Zapis znaleziska w raporcie 18
8. Pytania kontrolne 20

0. Wymagania

Do wykonania ćwiczenia potrzebne jest następujące oprogramowanie:

- **Node.js w wersji 18 lub nowszej** – pobierany ze strony nodejs.org. Node.js to środowisko uruchomieniowe języka JavaScript poza przeglądarką; służy do uruchomienia aplikacji ćwiczeniowej. Poprawność instalacji sprawdza polecenie `node --version`.
- **Burp Suite Community** – narzędzie pośredniczące w ruchu HTTP, opisane w ćwiczeniu 1. Służy do obejrzenia, powtórzenia i udokumentowania żądań ze wstrzykniętą wartością.
- **Przeglądarka** – wbudowana przeglądarka Chromium z Burp Suite, ta sama co w ćwiczeniu 1. Całe ćwiczenie wykonuje się w tej przeglądarce, dzięki czemu każde żądanie trafia do historii Burpa. Narzędzia deweloperskie otwiera się w niej klawiszem F12. Menu przeglądarki jest w języku systemu, a narzędzia deweloperskie zwykle w języku angielskim; nazwy elementów podano tak, jak wyświetla je przeglądarka, z odpowiednikiem w drugim języku w nawiasie.
- **Pliki aplikacji** – z katalogu ćwiczenia: `app.mjs`, `package.json` i `package-lock.json`.

Oprogramowanie z tej listy należy zainstalować przed zajęciami i sprawdzić, czy aplikacja uruchamia się poleceniami `npm install` oraz `npm start` (część 2). Redakcję znalezisk w formie raportu można dokończyć po zajęciach.

1. Jak działa XSS

Aplikacja webowa buduje stronę, wstawiając do kodu HTML wartości pochodzące od użytkownika: wpisaną frazę, treść opinii, imię z adresu. Jeśli wartość trafia do strony bez zmian, a zawiera znaki składni HTML, przeglądarka dołącza ją do strony jako kod, a nie jako tekst. Znacznik `<script>` zawarty w takiej wartości zostaje wykonany jak każdy inny skrypt strony.

Przyczyną jest brak kodowania wyjścia (ang. *output encoding*). Kodowanie wyjścia zamienia znaki składni HTML na encje znakowe (ang. *character entities*), czyli zapisy, które przeglądarka wyświetla jako pojedynczy znak. Zamiana obejmuje znaki `&` na `&`, `<` na `<`, `>` na `>`, `"` na `"` oraz `'` na `'`. Po zamianie przeglądarka wyświetla te znaki jako zwykły tekst i nie rozpoczyna od nich znacznika.

Podatny fragment aplikacji ćwiczeniowej wygląda następująco (język JavaScript):

```
const q = req.query.q ?? "";
res.send(`<p>Szukana fraza: ${q}</p>`);
```

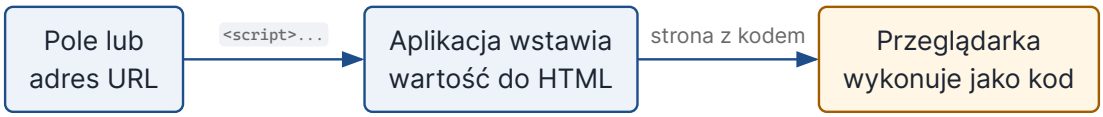
Zmienna `q` pochodzi z pola wyszukiwarki. Dla zwykłej frazy, na przykład wartości `Monitor`, przeglądarka otrzymuje poprawny fragment strony:

```
<p>Szukana fraza: Monitor</p>
```

Gdy użytkownik wpisze w pole wartość `<script>alert(1)</script>`, przeglądarka otrzymuje stronę z dodatkowym znacznikiem skryptu:

```
<p>Szukana fraza: <script>alert(1)</script></p>
```

Przeglądarka nie odróżnia tego skryptu od kodu napisanego przez autora strony i wykonuje go. Wartość z pola zostaje w ten sposób włączona do kodu strony, co stanowi istotę podatności.



Rysunek 1. Wartość od użytkownika zostaje wstawiona do treści strony. Przeglądarka otrzymuje ją jako część kodu HTML i wykonuje zawarty w niej skrypt.

Skutkiem XSS jest wykonanie kodu atakującego w przeglądarce innego użytkownika, w ramach jego sesji. Kod ma dostęp do tych samych danych co strona. Może odczytać ciasteczka sesji, wysłać żądania w imieniu zalogowanego użytkownika albo podmienić zawartość strony.

Trzy odmiany XSS różnią się tym, skąd pochodzi wartość i gdzie trafia:

Odmiana	Skąd pochodzi wartość i gdzie trafia
odbita (ang. *reflected*)	wartość z pojedynczego żądania (np. z adresu URL) wraca w odpowiedzi na to samo żądanie; atak wymaga nakłonienia ofiary do otwarcia spreparowanego adresu (część 3)
trwała (ang. *stored*)	wartość zostaje zapisana w aplikacji i wyświetlona każdemu, kto otworzy stronę; atak nie wymaga spreparowanego adresu (część 4)
oparta na modelu DOM (ang. *DOM-based*)	skrypt działający w przeglądarce odczytuje wartość i wstawia ją do strony, więc podatność jest w kodzie klienta (kodzie JavaScript wykonywanym w przeglądarce); w ćwiczeniu wartość pochodzi z fragmentu adresu po `#` (końcowej części adresu URL, której przeglądarka nie wysyła na serwer) (część 5)

Ciasteczko (ang. *cookie*) to krótka wartość, którą serwer zapisuje w przeglądarce i którą przeglądarka dołącza do każdego kolejnego żądania do tej samej aplikacji. Ciasteczko sesji identyfikuje zalogowanego użytkownika, więc osoba, która je pozna, może działać w aplikacji jako ten użytkownik. Skrypty strony odczytują ciasteczka przez właściwość `document.cookie`, chyba że serwer oznaczył ciasteczko atrybutem `HttpOnly` (część 6.3). W ćwiczeniu dowodem wykonania kodu jest wywołanie `alert(document.cookie)`. Funkcja `alert` wyświetla okno komunikatu z podanym tekstem, w tym przypadku z ciasteczkami bieżącej strony. Jest to nieszkodliwy sposób pokazania, że skrypt atakującego działa i ma dostęp do danych sesji. Wartość przygotowaną przez atakującego, która zawiera wstrzykiwany kod, nazywa się ładunkiem (ang. *payload*).

2. Uruchomienie aplikacji

W katalogu ćwiczenia (z plikiem `package.json`) wykonaj polecenia:

```
npm install
npm start
```

Pierwsze polecenie pobiera zależność (framework `express`), drugie uruchamia aplikację. Aplikacja nasłuchuje pod adresem `http://127.0.0.1:8000`. Okno terminala z uruchomioną aplikacją pozostaw otwarte do końca ćwiczenia.

Opinie dodawane w części 4 są przechowywane w pamięci procesu, dlatego zatrzymanie aplikacji (Ctrl+C) i ponowne uruchomienie przywracają stan początkowy. Aplikacja ustawia przy logowaniu ciasteczko sesji `session`, którego wartością jest sam login. Ciasteczko nie ma atrybutu `HttpOnly`, więc skrypty strony mogą je odczytać przez `document.cookie`.

Aplikacja udostępnia trzy podatne miejsca, każde z wersją poprawioną:

Adres	Opis
`GET /search?q=`	wyszukiwarka; fraza `q` wstawiana do strony bez kodowania (część 3)
`POST /reviews`	zapis opinii, parametry `author` i `text` (część 4)
`GET /reviews`	opinie o sklepie; podpis i treść opinii wyświetlane bez kodowania (część 4)
`GET /welcome`	powitanie; fragment adresu po `#` wstawiany przez `innerHTML`, czyli jako kod HTML (część 5)
`GET /fixed/search`	wersja poprawiona wyszukiwarki (część 6)
`GET /fixed/reviews`	wersja poprawiona listy opinii (część 6)
`GET /fixed/welcome`	wersja poprawiona powitania (część 6)

> **KROK**
>
> Uruchom Burp Suite. W zakładce *Proxy*, w podzakładce *Intercept*, upewnij się, że przechwytywanie jest wyłączone (*Intercept off*), i kliknij *Open browser*. Wszystkie kolejne kroki ćwiczenia wykonuje się w tej przeglądarce.
>
> Otwórz w niej adres `http://127.0.0.1:8000`, w polu *Login* pozostaw wartość `admin` i kliknij *Zaloguj*. Aplikacja ustawi ciasteczko `session=admin`, które odczyta wstrzyknięty skrypt. Bez logowania `document.cookie` jest puste, a okno komunikatu nie pokaże żadnej wartości.

Po zalogowaniu aplikacja wraca na stronę startową, a pod nagłówkiem sklepu pojawia się napis „Zalogowany jako admin”. Napis „Nie jesteś zalogowany” oznacza, że logowanie się nie powiodło i należy je powtórzyć.

> **UWAGA**
>
> Raport z testu opisuje jego przebieg, dlatego od części 3 należy zapisywać każdy sprawdzony adres lub żądanie, użytą wartość i wynik. Zrzuty ekranu wskazane w akapitach „Dowodem do raportu” należy wykonać od razu, ponieważ po ponownym uruchomieniu aplikacji zapisane opinie znikają.

3. XSS odbity (reflected)

Wyszukiwarka przyjmuje frazę w parametrze `q` w adresie URL i wstawia ją do strony z wynikami:

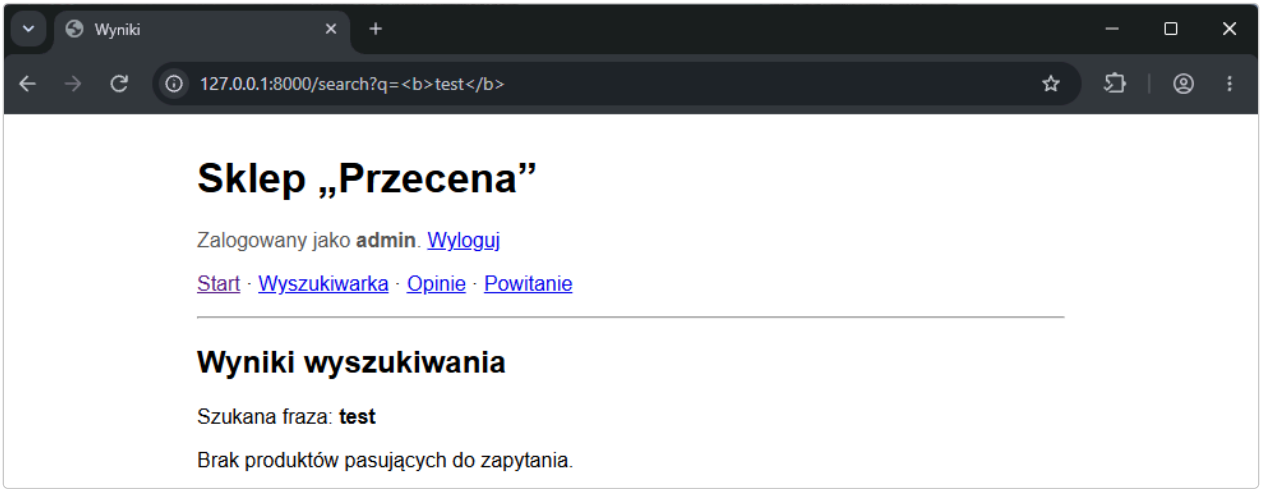
```
res.send(`<p>Szukana fraza: ${q}</p>`);
```

Wartość `q` trafia do strony bez kodowania. Parametry w adresie URL najprościej testuje się bezpośrednio w pasku adresu przeglądarki.

3.1. Potwierdzenie podatności

> **KROK**
>
> W pasku adresu przeglądarki wpisz adres, w którym fraza zawiera znacznik pogrubienia:
>
> `http://127.0.0.1:8000/search?q=<b>test</b>`

Na stronie słowo „test” zostanie pogrubione, a znaczniki nie będą widoczne. Oznacza to, że wartość z parametru została dołączona do strony jako kod HTML. Taki wynik wystarcza, aby podejrzewać XSS.



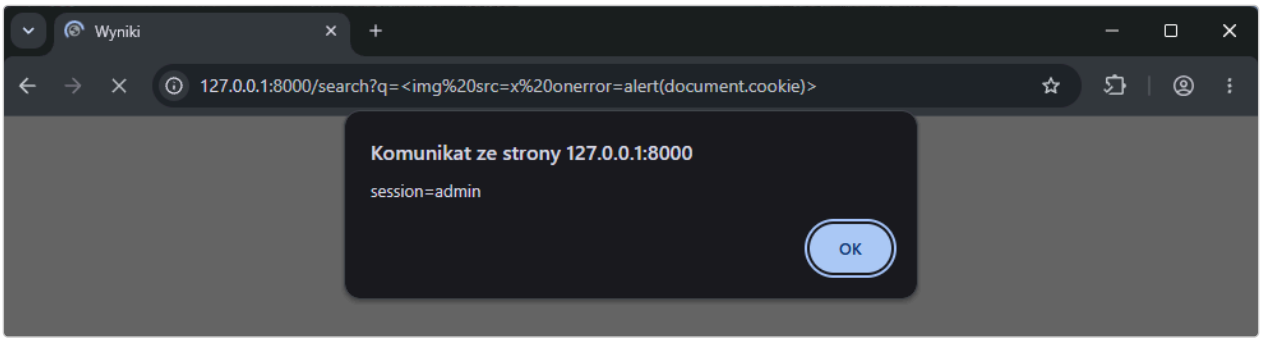
Zrzut 1. Wynik wyszukiwania dla frazy `<b>test</b>`. Słowo „test” jest pogrubione, a znaczniki nie są widoczne, ponieważ przeglądarka potraktowała je jako kod HTML.

3.2. Wykonanie skryptu

W wyszukiwarce zadziałałby znacznik `<script>` z części 1, ponieważ serwer wstawia go do kodu HTML strony. W ćwiczeniu stosuje się jednak jeden ładunek dla wszystkich trzech odmian, także dla części 5, w której znacznik `<script>` nie zostaje wykonany (przyczyna jest opisana w części 5). Ładunek wykorzystuje atrybut obsługi zdarzenia (taki jak `onerror`), którego wartość przeglądarka wykonuje jako kod JavaScript po wystąpieniu zdarzenia. Znacznik obrazka z nieistniejącym źródłem i atrybutem `onerror` wykonuje kod, gdy obrazka nie uda się wczytać:

> **KROK**
>
> W pasku adresu wpisz adres ze wstrzykniętym obrazkiem. Spacje w adresie zapisano jako `%20` (kodowanie procentowe, ang. *percent-encoding*, czyli zapis znaku jako `%` i dwóch cyfr szesnastkowych jego kodu; polska litera zajmuje dwa takie zapisy, np. ś to `%C5%9B`); przeglądarka zamienia je w ten sposób także wtedy, gdy adres zostanie wpisany ze spacjami.
>
> `http://127.0.0.1:8000/search?q=<img%20src=x%20onerror=alert(document.cookie)>`

Przeglądarka próbuje wczytać obrazek ze źródła `x`, nie znajduje go i wykonuje kod z atrybutu `onerror`. Pojawi się okno komunikatu z zawartością ciasteczka `session`. Oznacza to, że skrypt atakującego wykonał się na stronie aplikacji i odczytał ciasteczko sesji. Okno należy zamknąć przyciskiem *OK*; w miejscu frazy pozostanie ikona niewczytanego obrazka.



Zrzut 2. XSS odbity w wyszukiwarce. Wartość parametru `q` została wstawiona do strony jako kod; przeglądarka wykonała ją i wyświetliła ciasteczko sesji w oknie komunikatu.

W ataku odbitym wartość wraca tylko w odpowiedzi na to konkretne żądanie. Aby zaatakować inną osobę, atakujący przygotowuje powyższy adres i nakłania ofiarę do jego otwarcia, na przykład w wiadomości. Po otwarciu kod wykonuje się w przeglądarce ofiary, w ramach jej sesji.

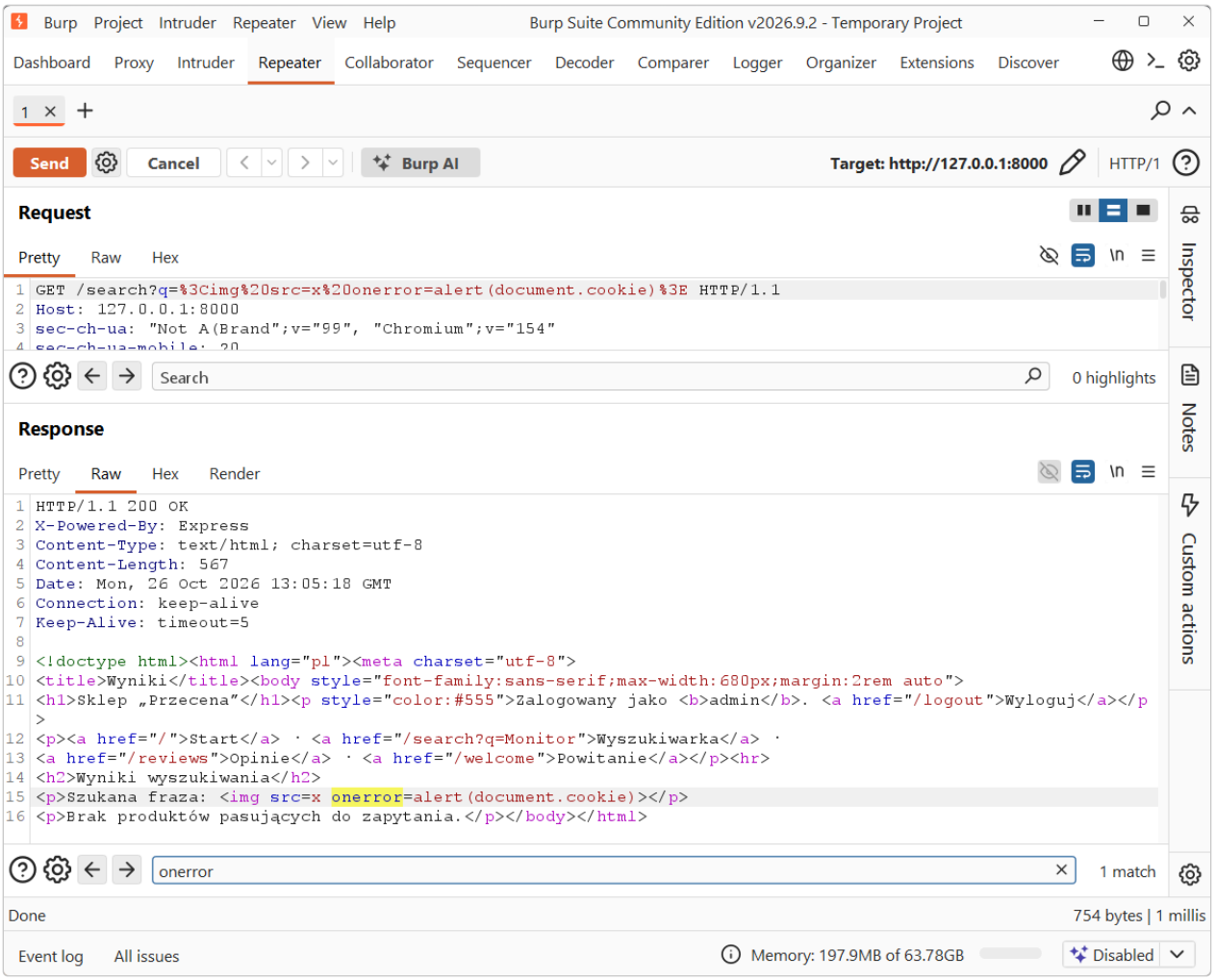
3.3. Dokumentowanie żądania w Burp Suite

Do raportu wygodnie jest zapisać samo żądanie ze wstrzykniętą wartością oraz fragment odpowiedzi, w którym widać wstrzyknięty kod. Poprzedni krok wykonano we wbudowanej przeglądarce Burpa, więc żądanie zostało już zapisane w jego historii.

> **KROK**
>
> W Burpie w zakładce *Proxy*, w podzakładce *HTTP history*, odszukaj żądanie `GET /search` ze wstrzykniętym obrazkiem. W kolumnie *URL* ma ono postać `/search?q=%3Cimg...`, ponieważ znaki `<` i `>` są zakodowane jako `%3C` i `%3E`; najnowsze żądania znajdują się na dole listy. Kliknij je prawym przyciskiem myszy i wybierz *Send to Repeater*. W zakładce *Repeater* naciśnij *Send*, a nad odpowiedzią wybierz widok *Raw*. Aby odnaleźć wstrzyknięty znacznik w odpowiedzi, wpisz `onerror` w polu wyszukiwania pod odpowiedzią.

W treści odpowiedzi znajduje się znacznik `<img ... onerror=...>` zwrócony bez zmian. W wierszu żądania (pierwszym wierszu, z metodą i adresem) przeglądarka zakodowała znaki `<` i `>` jako `%3C` i `%3E`, a spacje mają postać `%20` nadaną w adresie. Serwer dekoduje te zapisy przed wstawieniem wartości do strony.

Zaraz za żądaniem `/search` w historii znajduje się żądanie `GET /x`. Jest to próba wczytania obrazka ze źródła `x`, na którą serwer odpowiada kodem 404; nieudane wczytanie wywołuje zdarzenie `onerror`.



Zrzut 3. Żądanie `GET /search` ze wstrzykniętą wartością parametru `q` w zakładce *Repeater*. W odpowiedzi w widoku *Raw* znacznik `<img ... onerror=...>` został zwrócony bez kodowania (wyszukiwane słowo `onerror` jest podświetlone).

Dowodem do raportu jest wiersz żądania `GET /search` z wartością parametru `q` oraz fragment odpowiedzi, w którym widać wstrzyknięty znacznik. Oba fragmenty należy skopiować jako tekst z zakładki *Repeater* (zaznaczenie i Ctrl+C) i skrócić do istotnych wierszy, jak w przykładzie w części 7.

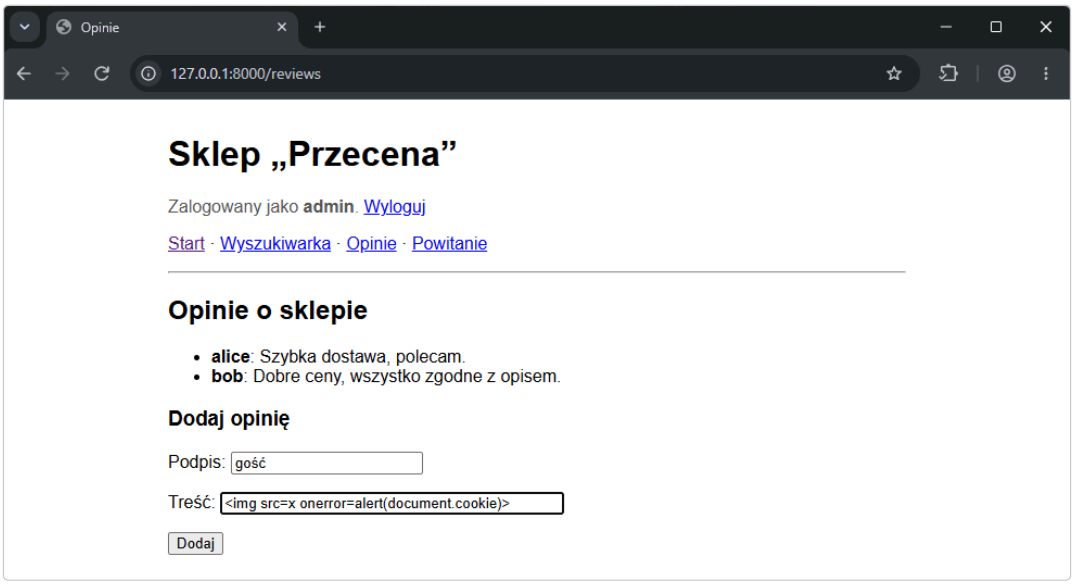
4. XSS trwały (stored)

Strona opinii przyjmuje podpis i treść opinii, zapisuje je w aplikacji i wyświetla listę wszystkim odwiedzającym:

```
reviews.push({ author, text });
// przy wyswietlaniu:
const items = reviews.map((r) => `<li><b>${r.author}</b>: ${r.text}</li>`).join("");
```

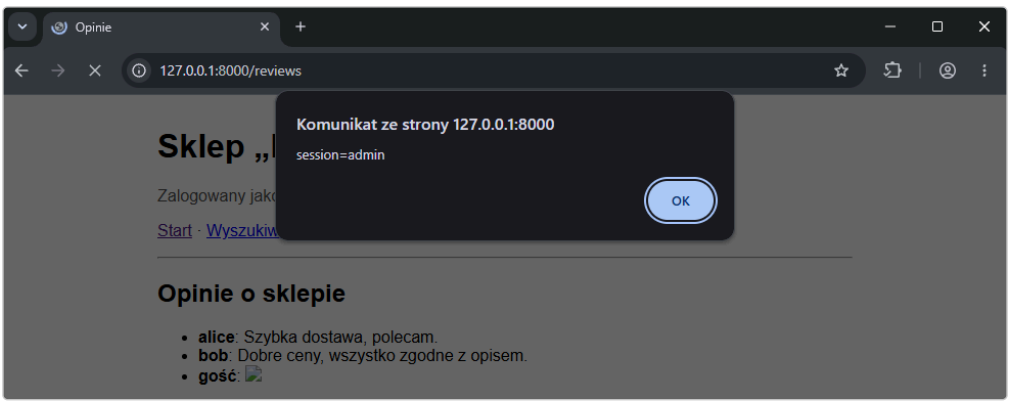
Podpis i treść opinii trafiają do strony bez kodowania. W odróżnieniu od ataku odbitego wartość zostaje zapisana, a zawarty w niej kod wykonuje się u każdego, kto otworzy stronę opinii, bez żadnego spreparowanego adresu.

> **KROK**
>
> Otwórz adres `http://127.0.0.1:8000/reviews`. W formularzu w polu *Treść* wpisz:
>
> `<img src=x onerror=alert(document.cookie)>`
>
> W polu *Podpis* pozostaw wartość domyślną `gość` i kliknij *Dodaj* tylko raz, ponieważ każde kliknięcie zapisuje kolejną opinię.



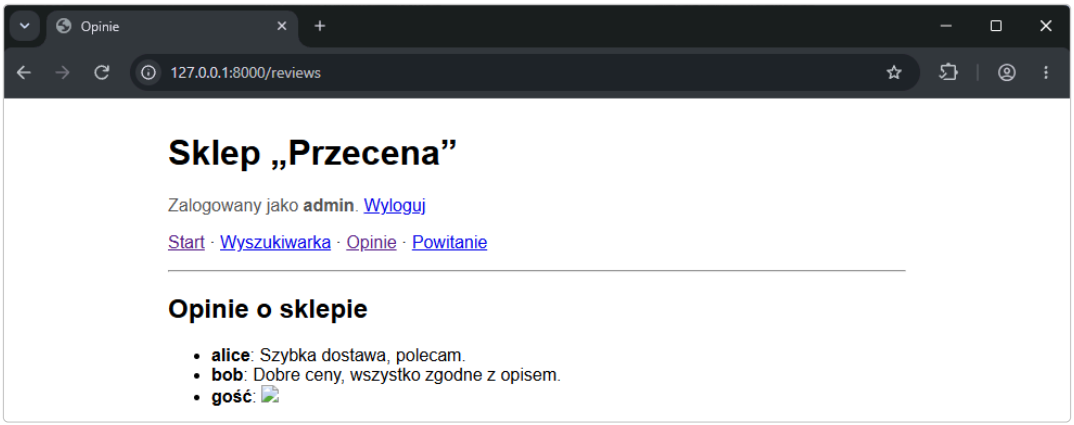
Zrzut 4. Formularz dodawania opinii z wpisanym w polu *Treść* znacznikiem obrazka z atrybutem `onerror`.

Po dodaniu opinii aplikacja przekieruje przeglądarkę na listę opinii. Lista zawiera już nową opinię, więc przeglądarka od razu wykona wstrzyknięty kod i wyświetli okno komunikatu z ciasteczkiem sesji.



Zrzut 5. XSS trwały w opiniach. Treść opinii została zapisana w aplikacji, a zawarty w niej skrypt wykonał się przy wyświetleniu listy, co potwierdza okno komunikatu.

Po zamknięciu okna komunikatu lista opinii wyświetla się w całości. W miejscu treści nowej opinii widoczna jest ikona obrazka, którego nie udało się wczytać. Przeglądarka utworzyła z treści opinii element `<img>`, zamiast wyświetlić ją jako tekst.



Zrzut 6. Lista opinii po zamknięciu okna komunikatu. Treść opinii podpisanej jako `gość` została wstawiona jako element `<img>` z nieistniejącym źródłem, dlatego widoczna jest tylko ikona niewczytanego obrazka.

> **KROK**
>
> Odśwież stronę opinii klawiszem F5 albo otwórz ją ponownie. Przed zamknięciem okna komunikatu wykonaj zrzut ekranu okna przeglądarki jako dowód do raportu.

Okno komunikatu pojawi się ponownie. Odmianę trwałą od odbitej odróżnia to, że kod wykonuje się za każdym razem, ponieważ opinia pozostaje zapisana. U każdego innego użytkownika, który otworzy stronę opinii, kod wykona się w jego przeglądarce i w ramach jego sesji. Jeśli opinie przegląda administrator sklepu, atak obejmuje także konta o wyższych uprawnieniach.

> **UWAGA**
>
> Kod zapisany w opinii wykonuje się przy każdym wejściu na `/reviews`. Opinia będzie potrzebna w części 6, dlatego należy ją pozostawić do czasu sprawdzenia wersji poprawionej. Później można ją usunąć, zatrzymując aplikację (Ctrl+C) i uruchamiając ją ponownie poleceniem `npm start`.

Treść opinii jest wysyłana w ciele żądania (ang. *request body*, części żądania po nagłówkach) jako parametr `text`, a podpis jako parametr `author`.

> **KROK**
>
> W Burpie w zakładce *Proxy*, w podzakładce *HTTP history*, zaznacz wiersz z metodą `POST` i adresem `/reviews` (zaraz za nim znajduje się żądanie `GET /reviews` wysłane po przekierowaniu). W panelu *Request* poniżej listy ostatni wiersz, oddzielony od nagłówków pustym wierszem, zawiera ciało żądania z parametrami `author` i `text`. Skopiuj wiersz żądania oraz ciało do notatek.

Wartości parametrów są zapisane w kodowaniu procentowym (np. `%3Cimg` zamiast `<img`; w treści formularza spacja jest zapisywana jako `+`). Zakodowane są także znaki `=`, `(` i `)` (`%3D`, `%28`, `%29`) oraz polskie litery z podpisu, dlatego w ciele żądania nie występuje dosłowny tekst `onerror=alert`.

Dowodem do raportu jest skopiowana treść żądania `POST /reviews` oraz zrzut ekranu listy opinii z oknem komunikatu, wykonany w poprzednim kroku. XSS trwały ma zazwyczaj wyższą wagę niż odbity, ponieważ nie wymaga nakłaniania ofiary do otwarcia spreparowanego adresu. Do wykonania kodu wystarcza otwarcie zwykłej strony aplikacji.

5. XSS oparty na modelu DOM (DOM-based)

Strona powitania wyświetla imię podane w adresie po znaku `#`. Odczytuje je skrypt działający w przeglądarce i wstawia do strony:

```
var name = decodeURIComponent(location.hash.slice(1));
if (name) document.getElementById("name").innerHTML = name;
```

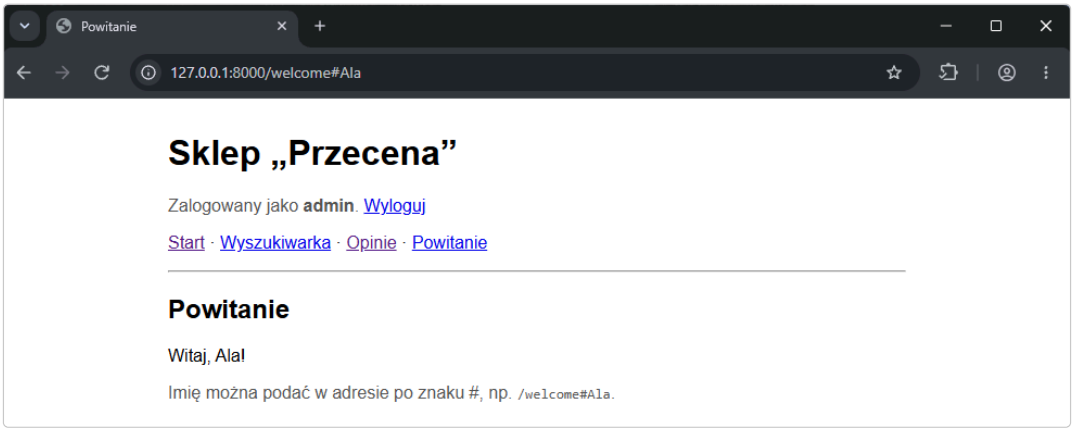
Właściwość `location.hash` zwraca fragment adresu razem ze znakiem `#`; wywołanie `slice(1)` usuwa ten znak, a `decodeURIComponent` dekoduje znaki zapisane w kodowaniu procentowym, np. `%20`. Przeglądarka zapisuje znaki `<`, `>` i spacje we fragmencie adresu jako `%3C`, `%3E` i `%20`, więc bez tego dekodowania wartość nie utworzyłaby znacznika. Pasek adresu wyświetla znaki `<` i `>` w postaci czytelnej, jednak właściwość `location.hash` zwraca je w postaci zakodowanej. Właściwość `innerHTML` ustawia zawartość elementu jako kod HTML. Skrypt pobiera więc wartość z adresu i wstawia ją do strony jako HTML, bez kodowania. Przeglądarka nie wykonuje znaczników `<script>` wstawionych przez `innerHTML`, dlatego ładunek korzysta z atrybutu `onerror`.

Fragment adresu po znaku `#` nie jest wysyłany na serwer, dlatego w tym przykładzie wartość nie przechodzi przez serwer. Serwer dostarcza skrypt strony, ale nie przetwarza wstrzykniętej wartości. Podatność znajduje się w całości w kodzie JavaScript wykonywanym w przeglądarce.



Rysunek 2. Wartość po znaku `#` nie dociera do serwera. Odczytuje ją skrypt strony i wstawia przez `innerHTML`, więc podatność w całości leży w kodzie wykonywanym w przeglądarce.

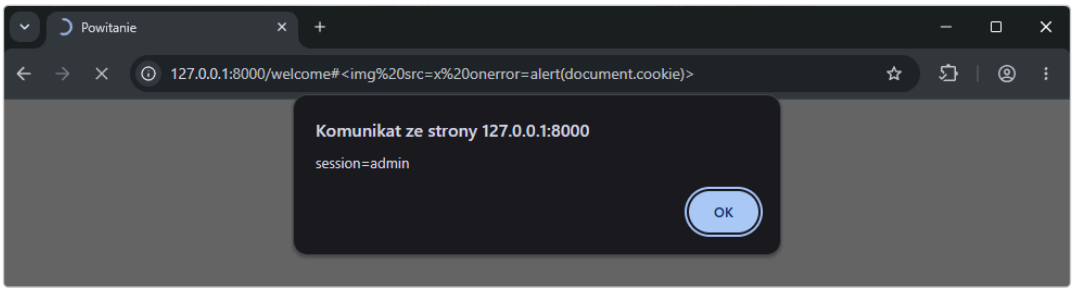
> **KROK**
>
> W pasku adresu wpisz adres z imieniem po znaku `#`:
>
> `http://127.0.0.1:8000/welcome#Ala`
>
> Strona wyświetli powitanie „Witaj, Ala!”.



Zrzut 7. Strona powitania dla adresu zakończonego fragmentem `#Ala`. Skrypt strony odczytał imię z fragmentu adresu i wstawił je do powitania.

> **KROK**
>
> Zmień fragment po `#` na wstrzyknięty znacznik:
>
> `http://127.0.0.1:8000/welcome#<img src=x onerror=alert(document.cookie)>`
>
> Zmiana samego fragmentu po `#` nie powoduje ponownego wczytania strony, a skrypt powitania wykonuje się tylko przy wczytaniu. Po wpisaniu adresu i naciśnięciu klawisza Enter odśwież stronę klawiszem F5.

Po odświeżeniu strony skrypt wstawia wartość przez `innerHTML`, przeglądarka próbuje wczytać obrazek, nie znajduje go i wykonuje kod z `onerror`. Pojawi się okno komunikatu z ciasteczkiem sesji.

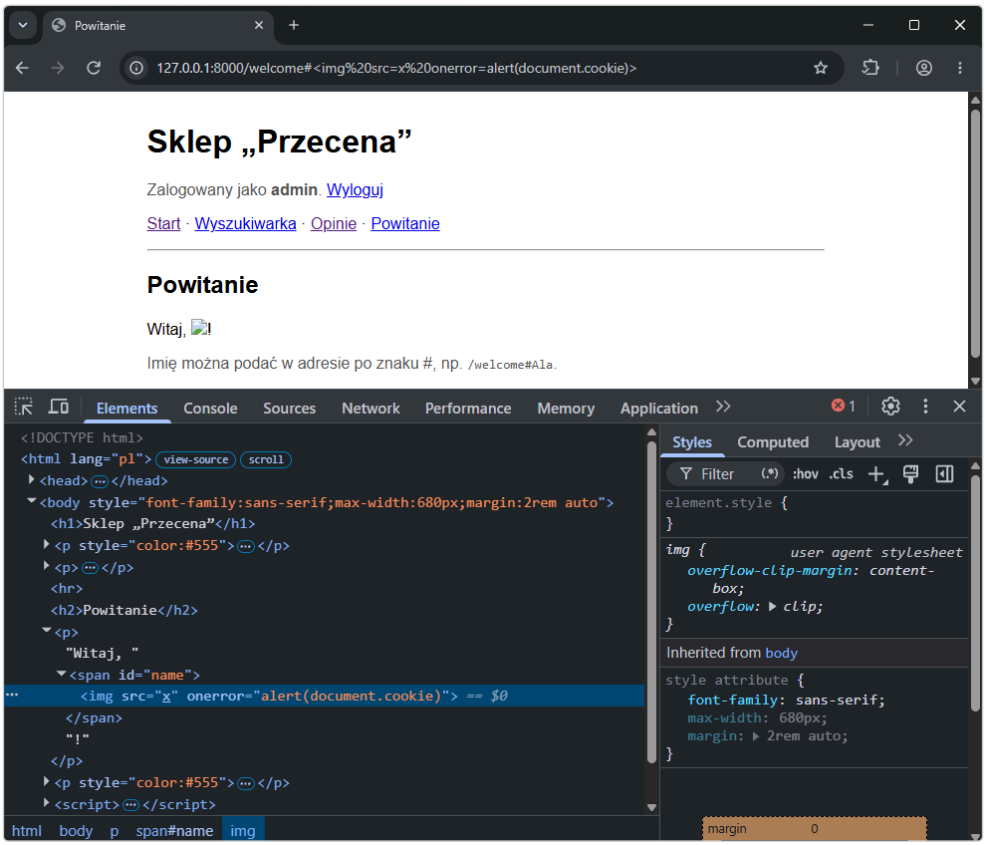


Zrzut 8. XSS oparty na modelu DOM na stronie powitania. Wartość z fragmentu adresu po `#` została wstawiona przez `innerHTML`, a zawarty w niej skrypt wykonał się w przeglądarce bez udziału serwera.

Efekt działania skryptu można obejrzeć w narzędziach deweloperskich przeglądarki.

> **KROK**
>
> Zamknij okno komunikatu. Kliknij prawym przyciskiem myszy małą ikonę niewczytanego obrazka za słowem „Witaj," i wybierz *Zbadaj* (*Inspect*).

Narzędzia deweloperskie otworzą się w zakładce *Elements* (*Elementy*) z zaznaczonym elementem `<img>`, który ma atrybut `onerror` i znajduje się wewnątrz elementu `<span id="name">`. Jeśli zaznaczony zostanie inny element, w zakładce *Elements* należy rozwinąć element `<p>` pod nagłówkiem *Powitanie* (trójkąt po lewej), a następnie element `<span id="name">`.



Zrzut 9. Zakładka *Elements* w narzędziach deweloperskich. Wewnątrz elementu `<span id="name">` znajduje się element `<img>` z atrybutem `onerror`, utworzony przez skrypt strony z fragmentu adresu.

> **KROK**
>
> Kliknij w obszar strony i naciśnij Ctrl+U. Źródło strony otworzy się w nowej karcie, której adres zaczyna się od `view-source:`. Wyszukaj w nim tekst `<img` (Ctrl+F), a następnie tekst `innerHTML`.

Wyszukiwanie tekstu `<img` nie zwróci żadnego wyniku, ponieważ serwer nie wysłał tego znacznika. Źródło strony zawiera tylko skrypt odczytujący fragment adresu, a wyszukiwanie tekstu `innerHTML` wskaże jego wiersz, który należy zapisać do raportu. Skrypt strony dodał element `<img>` w chwili przypisania wartości do `innerHTML`.

Dowodem do raportu jest adres ze wstrzykniętym fragmentem, wiersz kodu klienta, który wstawia wartość przez `innerHTML` (widoczny w źródle strony `/welcome`), oraz zrzut ekranu okna komunikatu albo zakładki *Elements* z elementem `<img>` wewnątrz `<span id="name">`. Dowód nie zawiera żądania do serwera ze wstrzykniętą wartością, ponieważ wartość nie opuszcza przeglądarki. W historii Burpa pojawią się żądania `GET /welcome` (bez fragmentu po `#`) oraz `GET /x`, ale żadne z nich nie zawiera wstrzykniętej wartości.

6. Poprawki

Wszystkie trzy odmiany mają tę samą przyczynę. Wartość od użytkownika trafia do strony tam, gdzie przeglądarka traktuje ją jako kod. Poprawka polega na tym, aby w miejscu wstawienia wartość była traktowana jako tekst. Właściwy sposób zależy od tego, dokąd wartość trafia (od kontekstu).

6.1. Kodowanie wyjścia w HTML (XSS odbity i trwały)

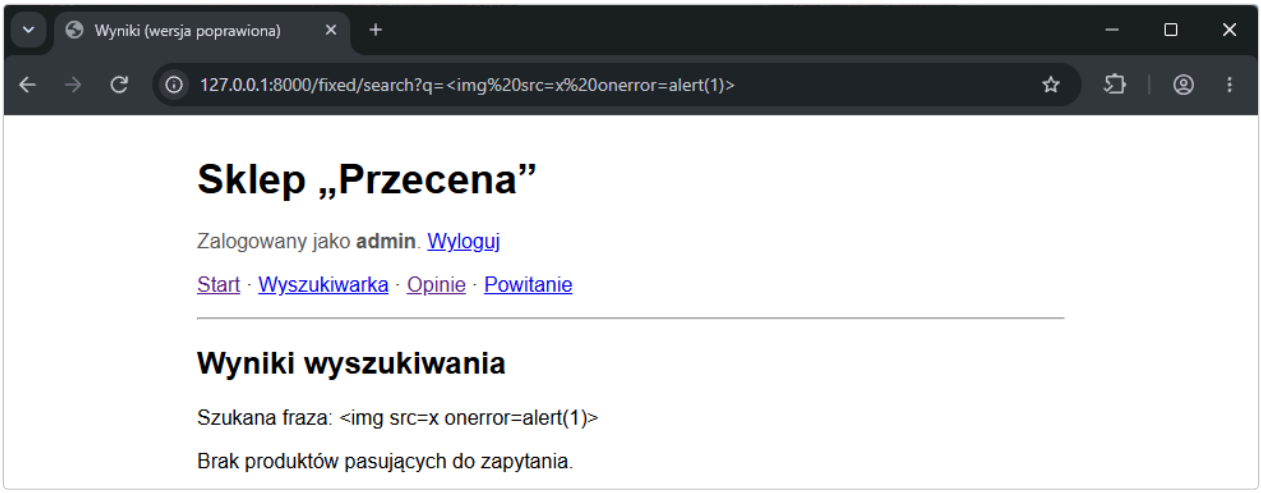
Gdy wartość wstawiana jest do treści HTML po stronie serwera, przed wstawieniem koduje się znaki składni HTML. W aplikacji ćwiczeniowej wersje poprawione `/fixed/search` oraz `/fixed/reviews` używają funkcji `esc`, która zamienia znaki `&`, `<`, `>`, `"` oraz `'` na encje znakowe (część 1):

```
const items = reviews.map((r) => `<li><b>${esc(r.author)}</b>: ${esc(r.text)}</li>`).join("");
```

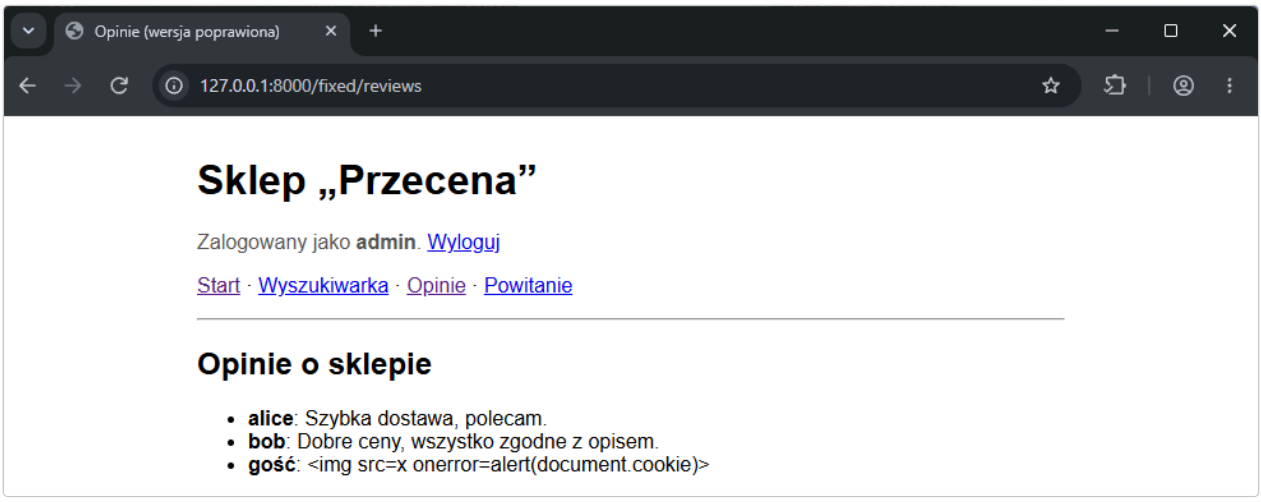
Po kodowaniu wartość `<img src=x onerror=...>` trafia do strony jako `<img src=x ...>`. Przeglądarka wyświetla ją jako tekst i nie rozpoczyna od niej znacznika. Takie kodowanie wystarcza w treści elementów i w wartościach zwykłych atrybutów ujętych w cudzysłów, np. `value` lub `title`. Nie chroni wartości wstawianej do kodu JavaScript, czyli do znacznika `<script>` lub do atrybutu obsługi zdarzenia (np. `onclick`), ani adresu w atrybucie `href` (np. `javascript:alert(1)`). Te konteksty wymagają innych zabezpieczeń.

> **KROK**
>
> Sprawdź oba ataki na wersjach poprawionych. Do sprawdzenia poprawki wystarczy dowolny kod, np. `alert(1)`. Odnośniki w menu aplikacji prowadzą do wersji podatnych, dlatego adresy wersji poprawionych wpisz w pasku adresu. Dla wyszukiwarki otwórz adres:
>
> `http://127.0.0.1:8000/fixed/search?q=<img%20src=x%20onerror=alert(1)>`
>
> Jeśli aplikacja była uruchamiana ponownie, dodaj najpierw opinię z części 4 na stronie `/reviews`. Następnie otwórz adres `http://127.0.0.1:8000/fixed/reviews`. Strona nie ma formularza i wyświetla te same opinie co `/reviews`, zakodowane przed wstawieniem.

Wstrzyknięty znacznik pojawi się na stronie jako zwykły tekst, a okno komunikatu nie zostanie wyświetlone. Ładunek nadal trafia do strony, ale w postaci encji znakowych, które przeglądarka wyświetla jako zwykłe znaki.



Zrzut 10. Wersja poprawiona wyszukiwarki. Wstrzyknięty znacznik jest wyświetlony jako zwykły tekst po słowach „Szukana fraza”.



Zrzut 11. Wersja poprawiona opinii. Ta sama wstrzyknięta opinia jest wyświetlona jako tekst, razem ze znakami `<` i `>`, a skrypt nie wykonuje się.

Do opisu przebiegu testu należy zapisać oba adresy wersji poprawionych, użytą wartość oraz wynik (znacznik wyświetlony jako tekst, brak okna komunikatu). Wynik ten potwierdza rekomendację w raporcie.

W praktyce kodowanie wykonują biblioteki szablonów (ang. *template engines*), które domyślnie kodują wstawiane wartości. Ręczne składanie kodu strony z fragmentów tekstu, jak w aplikacji ćwiczeniowej, pomija to zabezpieczenie i jest częstą przyczyną XSS.

6.2. Wstawianie jako tekst w kodzie klienta (XSS oparty na modelu DOM)

W XSS opartym na modelu DOM wartość do strony wstawia skrypt działający w przeglądarce. Serwer jej nie wstawia, więc kodowanie po stronie serwera nie ma tu zastosowania. Poprawka polega na wstawieniu wartości jako tekstu. Właściwość `textContent` ustawia zawartość elementu jako zwykły tekst, więc znaczniki w wartości nie stają się kodem. Skrypt wersji poprawionej `/fixed/welcome` różni się od podatnego jedną właściwością:

```
document.getElementById("name").textContent = name;
```

> **KROK**
>
> Powtórz atak na wersji poprawionej, wpisując w pasku adresu:
>
> `http://127.0.0.1:8000/fixed/welcome#<img src=x onerror=alert(1)>`
>
> Jeśli w karcie był już otwarty adres `/fixed/welcome` z innym fragmentem po `#`, odśwież stronę klawiszem F5.

W powitaniu pojawi się tekst znacznika, a okno komunikatu nie zostanie wyświetlone. Adres i wynik tego sprawdzenia należy zapisać tak samo jak dla wersji poprawionych wyszukiwarki i opinii.



Zrzut 12. Wersja poprawiona strony powitania. Wartość z fragmentu adresu została wstawiona przez `textContent`, więc znacznik jest widoczny jako tekst.

6.3. Zabezpieczenia uzupełniające

Poprawki opisane w częściach 6.1 i 6.2 usuwają przyczynę XSS. Dwa dodatkowe mechanizmy ograniczają skutki, gdy podatność jednak wystąpi:

- **Ciasteczko z atrybutem `HttpOnly`** nie jest dostępne dla skryptów strony, więc `document.cookie` go nie odczyta. Ogranicza to kradzież ciasteczka sesji, ale nie usuwa samej podatności, ponieważ skrypt atakującego nadal wykonuje się i może działać w imieniu użytkownika.
- **Nagłówek `Content-Security-Policy` (CSP)** to polityka, którą serwer wysyła wraz ze stroną i która określa między innymi, z jakich źródeł przeglądarka może wczytywać skrypty i czy może wykonywać skrypty osadzone w stronie (znaczniki `<script>` bez atrybutu `src` oraz atrybuty obsługi zdarzeń, takie jak `onerror`). Dobrze dobrana polityka blokuje wykonanie wstrzykniętego kodu. Stanowi jednak warstwę dodatkową i nie zastępuje kodowania wyjścia.

7. Zapis znaleziska w raporcie

Raport z testu penetracyjnego, poza listą znalezisk, opisuje przebieg testu, czyli to, które adresy i wartości sprawdzono i w jakiej kolejności. W tym ćwiczeniu przebieg obejmuje trzy sprawdzone miejsca (wyszukiwarkę, opinie i powitanie) oraz powtórzenie każdego ataku na wersji poprawionej.

Znalezisko opisuje się według następującego schematu:

1. Tytuł i identyfikator, np. „F-01. XSS odbity w wyszukiwarce produktów”.
2. Lokalizacja: endpoint (punkt końcowy, czyli metoda HTTP i ścieżka obsługiwane przez aplikację) oraz parametr, np. `GET /search`, parametr `q`.
3. Opis: na czym polega błąd i jaki skutek może wywołać.
4. Kategoria według OWASP Top 10 oraz numer CWE. OWASP Top 10 to publikowana przez fundację OWASP lista najpoważniejszych ryzyk bezpieczeństwa aplikacji webowych; oznaczenie `A05:2025` to kategoria 5 w wydaniu z 2025 roku. CWE (Common Weakness Enumeration) to katalog typów słabości oprogramowania. XSS należy do kategorii `A05:2025 Injection` oraz do `CWE-79` (Improper Neutralization of Input During Web Page Generation (’Cross-site Scripting’)).
5. Waga (krytyczna, wysoka, średnia, niska, według skali z ćwiczenia 1) z jednozdaniowym uzasadnieniem opisującym skutek dla właściciela systemu. W aplikacji ćwiczeniowej ciasteczko sesji nie ma atrybutu `HttpOnly`, więc wykonanie skryptu pozwala przejąć sesję ofiary. Przy ustalaniu wagi należy uwzględnić, czy atak wymaga dodatkowego działania ofiary, np. otwarcia spreparowanego adresu.
6. Kroki odtworzenia: numerowana lista czynności, która pozwala innej osobie powtórzyć test.
7. Dowód: adres lub żądanie ze wstrzykniętą wartością oraz fragment odpowiedzi, fragment kodu albo zrzut strony, na którym widać wykonanie kodu.
8. Rekomendacja: zmiana po stronie aplikacji, która usuwa przyczynę błędu.

Przykład dowodu dla XSS odbitego w wyszukiwarce:

```
GET /search?q=%3Cimg%20src=x%20onerror=alert(document.cookie)%3E HTTP/1.1
Host: 127.0.0.1:8000

HTTP/1.1 200 OK
...
<p>Szukana fraza: <img src=x onerror=alert(document.cookie)></p>
```

Dla znalezisk dotyczących XSS odbitego i trwałego rekomendacją jest kodowanie wartości przed wstawieniem do treści HTML (zamiana znaków składni HTML na encje znakowe), jak w wersjach `/fixed/search` oraz `/fixed/reviews`. Dla znaleziska dotyczącego XSS opartego na modelu DOM rekomendacją jest wstawienie wartości jako tekstu (`textContent`) zamiast jako kodu (`innerHTML`), jak w wersji `/fixed/welcome`. Jako zabezpieczenie uzupełniające można zalecić oznaczenie ciasteczka `session` atrybutem `HttpOnly` oraz wysyłanie nagłówka `Content-Security-Policy` (część 6.3), z zaznaczeniem, że nie zastępują one tych poprawek.

Znalezisko dotyczące XSS trwałego obejmuje dwa endpointy: wartość jest zapisywana przez `POST /reviews` (parametr `text`), a wykonywana przy wyświetleniu listy przez `GET /reviews`; w polu lokalizacji należy podać oba. Parametr `author` jest wstawiany do strony w tym samym wierszu kodu, również bez kodowania (fragment kodu w części 4); w raporcie można go wymienić z adnotacją, że wniosek wynika z analizy kodu, a nie z testu.

W znalezisku dotyczącym XSS opartego na modelu DOM lokalizacją jest strona `/welcome`, źródło wartości `location.hash` oraz wiersz skryptu, który przypisuje wartość do `innerHTML`. Dowodem jest ten wiersz kodu przepisany ze źródła strony oraz adres użyty w teście.

Ciasteczka innych użytkowników użyte jako dowód należy ograniczyć do minimum, zamaskować w raporcie i usunąć po zakończeniu testu. W ćwiczeniu ciasteczko `session=admin` pochodzi z własnego konta testowego i nie wymaga maskowania.

Po zakończeniu ćwiczenia notatki powinny zawierać materiał do trzech znalezisk:

Znalezisko	Lokalizacja	Dowód i sprawdzenie poprawki
F-01. XSS odbity	`GET /search`, parametr `q`	wiersz żądania i fragment odpowiedzi z zakładki *Repeater* (część 3); wynik dla `/fixed/search`
F-02. XSS trwały	`POST /reviews`, parametr `text`; `GET /reviews`	treść żądania `POST` i zrzut listy opinii (część 4); wynik dla `/fixed/reviews`
F-03. XSS oparty na modelu DOM	`GET /welcome`, źródło wartości `location.hash`	adres użyty w teście, wiersz kodu z `innerHTML` ze źródła strony i zrzut okna komunikatu (część 5); wynik dla `/fixed/welcome`

8. Pytania kontrolne

1. Dlaczego fraza `<b>test</b>` w wyszukiwarce jest wyświetlana jako pogrubione słowo, a nie jako tekst ze znacznikami?
2. Dlaczego w ćwiczeniu zastosowano znacznik obrazka z atrybutem `onerror` zamiast znacznika `<script>`?
3. Na czym polega różnica między XSS odbitym a trwałym z punktu widzenia sposobu dotarcia do ofiary?
4. Dlaczego XSS oparty na modelu DOM na stronie `/welcome` nie zostawia śladu w żądaniu wysłanym na serwer?
5. Czym różni się `innerHTML` od `textContent` i dlaczego ta różnica usuwa podatność?
6. Dlaczego kodowanie wyjścia po stronie serwera nie naprawia XSS opartego na modelu DOM?
7. Co ogranicza atrybut `HttpOnly` ciasteczka, a czego nie usuwa?

> **DALSZA PRAKTYKA**
>
> Do dalszych ćwiczeń można wykorzystać PortSwigger Web Security Academy (bezpłatne laboratoria z XSS, z ćwiczeniem i rozwiązaniem dla każdej odmiany) oraz aplikację OWASP Juice Shop. Testy należy prowadzić wyłącznie we własnym środowisku lub za pisemną zgodą właściciela systemu.